通用变换器

Mostafa Dehghani[*†]
University of Amsterdam
dehghani@uva.nl

Stephan Gouws[*]
DeepMind
sgouws@google.com

Oriol Vinyals
DeepMind
vinyals@google.com

Jakob Uszkoreit
Google Brain
usz@google.com

Łukasz Kaiser
Google Brain
lukaszkaiser@google.com

arXiv:1807.03819v3 [cs.CL] 5 Mar 2019

摘要

循环神经网络（RNN）通过在每个新数据点更新其状态来顺序处理数据，长期以来一直是序列建模任务的事实标准选择。然而，其固有的顺序计算特性导致训练速度缓慢。前馈和卷积架构最近已被证明在机器翻译等某些序列建模任务上取得了更优的结果，其额外优势在于能够同时处理序列中的所有输入，从而易于并行化并缩短训练时间。尽管取得了这些成功，但流行的前馈序列模型（如变换器）在许多简单任务中仍无法泛化，而循环模型却能轻松处理这些任务，例如复制字符串，甚至在字符串或公式长度超过训练时观察到的长度时进行简单的逻辑推理。我们提出了通用变换器（UT），这是一种时间上并行的自注意力循环序列模型，可视为变换器模型的推广，并解决了上述问题。UT 结合了变换器等前馈序列模型的并行性和全局感受野，以及 RNN 的循环归纳偏置。我们还添加了动态的逐位置停止机制，并发现它在多项任务上提高了准确率。与标准变换器相比，在某些假设下，UT 可以被证明是图灵完备的。我们的实验表明，UT 在广泛的算法和语言理解任务上优于标准变换器，包括具有挑战性的 LAMBADA 语言建模任务（UT 在该任务上达到了新的最新技术水平），以及机器翻译任务（UT 在 WMT14 英德数据集上比变换器提高了 0.9 BLEU 分数）。

1 引言

卷积和全注意力前馈架构（如变换器）近来已成为循环神经网络（RNN）在一系列序列建模任务中的可行替代方案，尤其是在机器翻译领域（Gehring 等，2017；Vaswani 等，2017）。这些时间并行架构解决了 RNN 的一个显著缺陷，即其固有的顺序计算阻碍了输入序列各元素间的并行化，同时仍能应对序列长度增加时的梯度消失问题（Hochreiter 等，2003）。特别是变换器模型完全依赖自注意力机制（Parikh 等，2016；Lin 等，2017）来计算其输入和输出中符号的一系列上下文感知的向量空间表示，随后在模型逐符号预测输出序列时，利用这些表示预测后续符号的分布。该机制不仅易于并行化，而且由于每个符号的表示也直接受所有其他符号表示的影响，从而在整个序列上形成有效的全局感受野。这与通常仅具有有限感受野的卷积架构形成对比。

然而值得注意的是，具有固定不同层堆叠的变换器放弃了 RNN 对学习迭代或递归变换的归纳偏置。我们的实验表明，这种归纳

[*] 同等贡献，按姓氏字母顺序排列。
[†] 在谷歌大脑期间完成的工作。

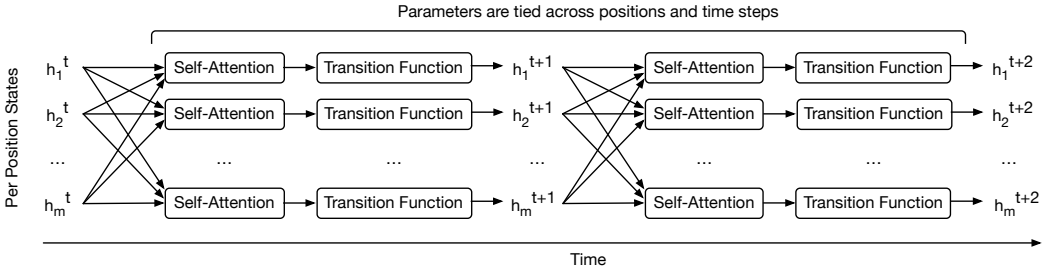


图 1：通用变换器通过使用自注意力（见公式 2）组合不同位置的信息，并在所有时间步上应用循环转移函数（见公式 4），并行地反复细化序列中每个位置的一系列向量表示 $1 \leq t \leq T$ 。我们展示了两个循环时间步上的这一过程。箭头表示操作之间的依赖关系。最初， h^0 用序列中每个符号的嵌入进行初始化。 h_i^t 表示输入符号 $1 \leq i \leq m$ 在循环时间步 t 的表示。通过动态停止， T 为每个位置动态确定（第 2.2 节）。

偏差对于若干不同复杂度的算法和语言理解任务可能至关重要：与神经图灵机（Neural Turing Machine）（Graves 等，2014）、神经 GPU（Neural GPU）（Kaiser 与 Sutskever，2016）或堆栈循环神经网络（Stack RNNs）（Joulin 与 Mikolov，2015）等模型相比，变换器（Transformer）在训练期间未遇到的输入长度上泛化能力不佳。

本文提出通用变换器（Universal Transformer，UT），一种时间上并行的循环自注意力序列模型，可视为变换器模型的推广，从而提升理论能力，并在众多具有挑战性的序列到序列任务上取得更优结果。通用变换器结合了前馈序列模型（如变换器）的并行性与全局感受野，以及循环神经网络（RNN）的循环归纳偏置，后者似乎更适合一系列算法与自然语言理解的序列到序列问题。顾名思义，与标准变换器不同，在特定假设下，通用变换器可被证明是图灵完备的（或“计算通用”，见第 4 节）。

在每个循环步骤中，通用变换器利用自注意力机制（Parikh 等，2016；Lin 等，2017）并行地迭代细化序列中所有符号的表示，随后进行一个变换（在所有位置和时间步上共享），该变换由深度可分离卷积（Chollet，2016；Kaiser 等，2017）或逐位置全连接层组成（见图 1）。本文还添加了动态逐位置停止机制（Graves，2016），使模型能够为每个符号动态选择所需的细化步数，并首次证明这种条件计算机制确实能够提高若干较小规模、结构化的算法和语言推理任务的准确率（尽管在机器翻译上略有性能下降）。

本文强有力的实验结果表明，通用变换器在广泛的任务上优于变换器和长短期记忆网络（LSTM）。增加的循环结构在机器翻译中带来了更好的结果，通用变换器超越了标准变换器。在若干算法任务和 bAbI 语言理解任务的实验中，通用变换器也一致且显著地优于长短期记忆网络和标准变换器。此外，在具有挑战性的 LAMBADA 文本理解数据集上，采用动态停止机制的通用变换器达到了新的最新技术水平。

2 模型描述

2.1 通用变换器

通用变换器（UT；见图 2）基于大多数神经序列到序列模型中常用的编码器-解码器架构（Sutskever 等，2014；Cho 等，2014；Vaswani 等，2017）。UT 的编码器和解码器分别通过对输入和输出序列中每个位置的表示应用循环神经网络来运行。然而，与循环神经网络在序列数据上的大多数应用相比，UT 不在序列中的位置上循环，而是在每个位置向量表示的连续修订上循环（即在“深度”上）。换句话说，UT 的计算不受序列中符号数量的限制，而仅受对每个符号表示所做修订次数的限制。

在每个循环时间步中，每个位置的表示同时（并行地）在两个子步骤中被修订：首先，使用自注意力机制在序列中的所有位置之间交换信息，从而为每个位置生成一个向量表示，该表示由前一时间步所有其他位置的表示所告知。然后，通过对自注意力机制的输出应用一个转移函数（在位置和时间上共享），在每个位置独立进行。由于循环转移函数可以应用任意次数，这意味着 UT 可以具有可变深度（每个符号的处理步骤数）。关键的是，这与大多数流行的神经序列模型（包括变换器（Vaswani 等，2017）或深度循环神经网络）形成对比，后者由于应用固定的层堆栈而具有恒定深度。我们现在更详细地描述编码器和解码器。

编码器： 给定长度为 m 的输入序列，我们从一个矩阵开始，其行初始化为序列 $H^0 \in \mathbb{K}^{m\times d}$ 中每个位置符号的 d 维嵌入。然后，UT 通过应用 Vaswani 等（2017）的多头点积自注意力机制，随后是循环转移函数，在步骤 t 为所有 m 个位置并行地迭代计算表示 H^t 。我们还在这些函数块周围添加残差连接，并应用随机失活和层归一化（Srivastava 等，2014；Ba 等，2016）（简化图见图 2，完整模型见附录 A 中的图 4）。

更具体地说，我们使用缩放点积注意力，它将查询 Q 、键 K 和值按如下方式结合 values V

$$\textsc{Attention}(Q,K,V) = \textsc{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \tag{1}$$

其中 d 是 Q, K 和 V 的列数。我们使用多头版本，具有 k 个头，如（Vaswani et al., 2017）中所介绍，

$$\textsc{MultiHeadSelfAttention}(H^t) = \textsc{Concat}(\text{head}_1,...,\text{head}_\text{k})W^O \tag{2}$$

$$\text{where head}_\text{i} = \textsc{Attention}(H^tW_i^Q, H^tW_i^K, H^tW_i^V) \tag{3}$$

并且我们使用学习到的参数矩阵 $W^Q \in \mathbb{R}^{d\times d/k}, W^K \in \mathbb{R}^{d\times d/k}, W^V \in \mathbb{R}^{d\times d/k}$ 和 $W^O \in \mathbb{R}^{d\times d}$. 通过仿射投影将状态 H^t 映射到查询、键和值。在第 t 步，通用变换器（UT）随后为所有 m 个输入位置计算修正表示 $H^t \in \mathbb{K}^{m\times d}$ ，如下所示

$$H^t = \textsc{LayerNorm}(A^t + \textsc{Transition}(A^t)) \tag{4}$$

$$\text{where } A^t = \textsc{LayerNorm}((H^{t-1}+P^t) + \textsc{MultiHeadSelfAttention}(H^{t-1}+P^t)), \tag{5}$$

其中层归一化（LAYERNORM()）在 Ba et al. (2016) 中定义，过渡函数（TRANSITION()）和 P^t 将在下文讨论。

根据任务的不同，本文采用两种不同的转移函数之一：要么是可分离卷积（Chollet, 2016），要么是一个全连接神经网络，该网络在两个仿射变换之间包含一个单一的整流线性激活函数，按位置应用，即 对上述 $P^t \in \mathbb{K}^{m\times d}$ 的每一行分别施加固定的、恒定的二维（位置，时间）坐标嵌入，其计算方式为：针对位置 $1 \le i \le m$ 和时间步 $1 \le t \le T$ ，分别对每个向量维度 $1 \le j \le d$, 计算（Vaswani 等人，2017）中定义的正弦位置嵌入向量，然后求和：A^t.

$$P^t_{i,2j} = \sin(i/10000^{2j/d}) + \sin(t/10000^{2j/d}) \tag{6}$$

$$P^t_{i,2j+1} = \cos(i/10000^{2j/d}) + \cos(t/10000^{2j/d}). \tag{7}$$

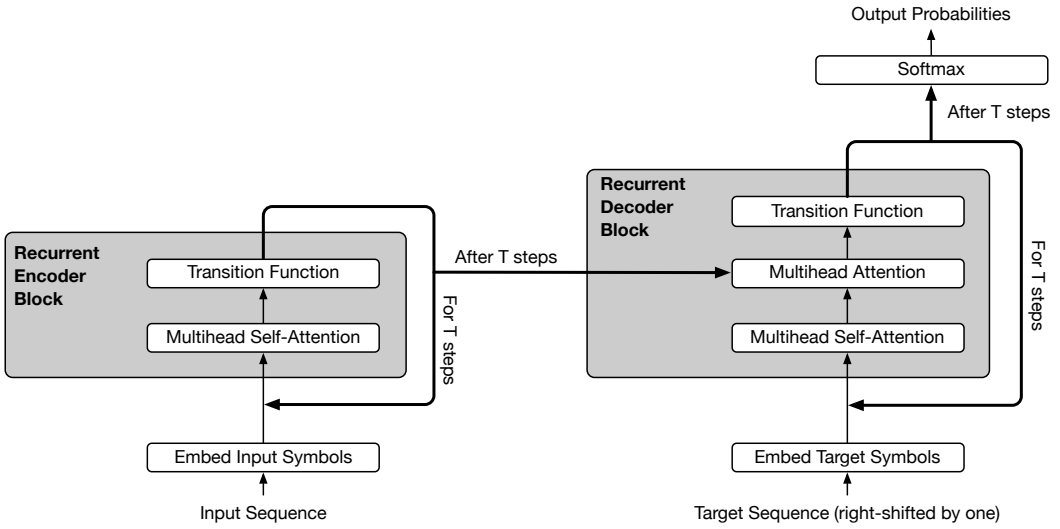


图 2：通用变换器编码器和解码器的循环块。此图省略了位置和时间步编码以及随机失活、残差连接和层归一化。完整版本见附录 A。具有动态停止的通用变换器使用 ACT（Graves, 2016）为每个位置单独确定步数 T 。

经过 T 步（每步并行更新输入序列的所有位置）后，通用变换器编码器的最终输出是一个矩阵，包含输入序列的 m 个符号的 d 维向量表示 $H^T \in \mathbb{R}^{m \times d}$ 。

D 解码器：解码器与编码器共享相同的基本循环结构。然而，在自注意力函数之后，解码器还利用公式 2 中相同的多头点积注意力函数，额外关注输入序列中每个位置的最终编码器表示 H^T ，但查询 Q 来自解码器表示的投影，键和值（ K 和 V ）来自编码器表示的投影（此过程类似于标准注意力机制（**Bahdanau** 等人，2014 年））。

与变换器模型类似，通用变换器是自回归的（Graves，2013 年）。在训练时使用教师强制，在生成时逐符号产生输出，解码器消费先前生成的输出位置。在训练期间，解码器输入是目标输出，向右移动一个位置。解码器自注意力分布进一步被掩码，使得模型只能关注任何预测符号左侧的位置。最后，通过从最终解码器状态到输出词汇表大小 V 的仿射变换 $O \in \mathbb{R}^{d \times V}$ ，然后经过 Softmax 函数，得到每个符号的目标分布，该 Softmax 函数产生一个 $(m \times V)$ 维输出矩阵，并在其行上进行归一化：

$$p\left(y_{pos} | y_{[1:pos-1]}, H^T\right) = \text{SOFTMAX}(OH^T)^1 \tag{8}$$

为了从模型生成，编码器对条件输入序列运行一次。然后解码器重复运行，消费所有已生成的符号，同时每次迭代为下一个输出位置的符号生成一个额外的词汇表分布。我们通常采样或选择概率最高的符号作为下一个符号。

2.2 动态停止

在序列处理系统中，某些符号（例如某些单词或音素）通常比其他符号更模糊。因此，为这些更模糊的符号分配更多处理资源是合理的。自适应计算时间（ACT）（Graves，2016 年）是一种动态调节处理每个输入符号所需的计算步骤数的机制

1 注意，这里的 T 表示时间步 T ，而不是转置操作。

Model	10K examples		1K examples	
	train single	train joint	train single	train joint
Previous best results:				
QRNet (Seo et al., 2016)	0.3 (0/20)	-	-	-
Sparse DNC (Rae et al., 2016)	-	2.9 (1/20)	-	-
GA+MAGE Dhingra et al. (2017)	-	-	8.7 (5/20)	-
MemN2N Sukhbaatar et al. (2015)	-	-	-	12.4 (11/20)
Our Results:				
Transformer (Vaswani et al., 2017)	15.2 (10/20)	22.1 (12/20)	21.8 (5/20)	26.8 (14/20)
Universal Transformer (this work)	0.23 (0/20)	0.47 (0/20)	5.31 (5/20)	8.50 (8/20)
UT w/ dynamic halting (this work)	**0.21 (0/20)**	**0.29 (0/20)**	**4.55 (3/20)**	**7.78 (5/20)**

表 1：在 bAbI 数据集上，不同训练/评估设置下的平均错误率和失败任务数（ $>5\%$ 错误）在 20 个任务中（括号内；两者均越低越好）。我们为每个设置标出可用的最先进结果，否则标为“-”。

（称为“思考时间”）在标准循环神经网络中，基于模型每一步预测的标量停止概率。

受通用变换器可视为对序列中所有位置并行应用自注意力循环神经网络这一解释的启发，我们还在每个位置（即每个符号的自注意力循环神经网络；详见附录 C）添加了动态自适应计算时间停止机制。一旦每个符号的循环块停止，其状态便直接复制到下一步，直到所有块停止或达到最大步数。编码器的最终输出即为由此产生的最后一层表示。

3 实验与分析

我们在多种算法和语言理解任务以及机器翻译上评估了通用变换器。这些任务和数据集的详细描述见附录 D。

3.1 BABI 问答

bAbi 问答数据集（Weston 等，2015）包含 20 个不同任务，目标是在给定若干英文句子（可能编码多个支持事实）的情况下回答问题。其目的是通过要求对每个故事中呈现的语言事实进行特定类型的推理来衡量各种形式的语言理解。标准变换器在此任务上表现不佳 [2] 。然而，我们基于通用变换器设计的模型在该任务上取得了最先进的结果。

为编码输入，与 Henaff 等（2016）类似，我们首先对故事中的每个事实应用学习到的乘法位置掩码到每个词的嵌入上，并将所有嵌入求和。问题以相同方式嵌入，然后将这些事实和问题的嵌入输入（通用）变换器。

如最初提出的那样，模型可以分别在每个任务上训练（“单独训练”）或在所有任务上联合训练（“联合训练”）。表 1 总结了我们的结果。我们进行了 10 次不同初始化的运行，并根据验证集上的性能选择最佳模型，与先前工作类似。通用变换器（UT）和带动态停止的 UT 在所有任务上均取得了最先进的平均错误率和失败任务数 [3] ，在 10K 和 1K 训练设置下均如此（按任务细分见附录 E）。

为了更好地理解模型的工作机制，我们分析了该任务的注意力分布和平均自适应计算时间（ACT）停留时间（详见附录 F）。首先，我们观察到注意力分布起初非常均匀，但在后续步骤中逐渐聚焦于回答每个问题所需的正确支持事实上，这与人类解决该任务的方式非常相似。其次，通过动态停止机制，我们观察到平均停留时间（即深度 2 我们尝试了不同的超参数和网络规模，但总是过拟合。3 定义为 $>5\%$ 误差。

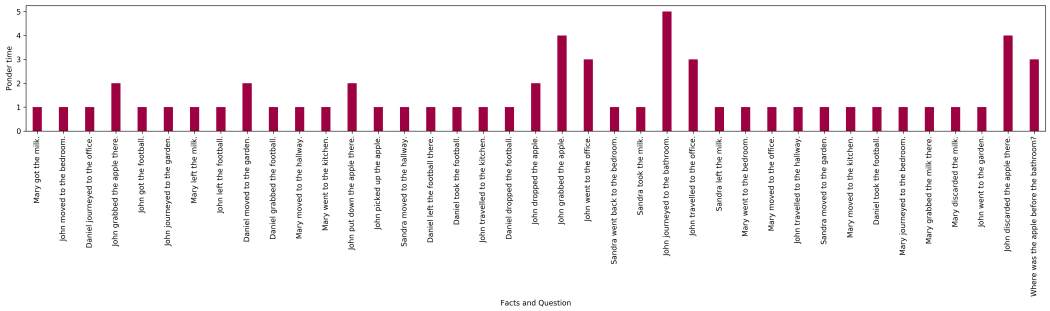


图 3：在需要三个支持事实的 bAbI 任务中，使用动态停止的 UT 对故事中的事实和问题进行编码时的停留时间。

在测试数据中，需要三个支持事实的任务在所有位置上的逐符号循环处理链的平均停留时间（ 3.8±2.2 ）高于仅需要两个支持事实的任务（ 3.1±1.1 ），而后者又高于仅需要一个支持事实的任务（ 2.3±0.8 ）。这表明模型会根据回答问题所需支持事实的数量来调整处理步骤数。最后，我们观察到，在仅需要一个支持事实的任务中，不同位置的停留时间直方图比需要两个和三个支持事实的任务更均匀，需要两个支持事实的任务也比需要三个的更均匀。特别是在需要三个支持事实的任务中，许多位置在第 1 步或第 2 步就已停止，只有少数位置会进行更多步的变换（例如见图 3 ）。这一点尤其有趣，因为在此设置下故事长度确实更长，包含更多无关事实，而模型似乎成功地学会了以这种方式忽略它们。

与动态记忆网络（Kumar 等人，2016）类似，UT 中存在迭代注意力过程，使模型能够根据先前迭代的结果来调整对记忆的注意力。附录 F 给出了一些示例，说明 UT 中存在时间状态的概念，模型在每一步根据前一步的输出更新其状态（记忆），这一更新链也可视为多跳推理过程中的步骤。

3.2 主谓一致

接下来，我们考虑预测英语句子中主语与动词之间数的一致性的任务（Linzen 等人，2016）。该任务作为衡量模型捕捉自然语言句子中层次（依存）结构能力的代理指标。我们使用（Linzen 等人，2016）提供的数据集，并遵循他们的实验协议，通过语言建模训练设置（即下一个词预测目标）来解决任务，然后在测试时计算目标动词的排序准确率。我们在测试数据的不同子集上评估了我们的模型，这些子集具有不同的任务难度，以一致性吸引子来衡量——即与主语数相反的中间名词的数量（旨在迷惑模型）。例如，给定句子“The keys to the cabinet [4] ”，训练期间的目标是预测动词 are （复数）。在测试时，我们评估一致性吸引子的排序准确率：即在这种情况下，目标是将 are 排在 is 之前。

我们的结果总结在表 2 中。文献中最好的带注意力的 LSTM 在此任务上达到 99.18%（Yogatama 等人，2018），优于普通 Transformer（Tran 等人，2018）。UT 显著优于标准 Transformer，并达到与当前最先进水平相当的平均结果（99.2%）。然而，我们看到随着吸引子数量的增加，UT（尤其是动态停止的 UT）的表现逐渐优于所有其他模型（见最后一行，Δ).

3.3 LAMBADA 语言建模

LAMBADA 任务（Paperno 等人，2016）是一项语言建模任务，要求在给定 4-5 个前句的广泛上下文中预测缺失的目标词。该数据集经过专门设计，使得人类在显示完整上下文时能够准确预测目标词，但在仅显示目标词所在的句子时则无法预测。因此，它超越了语言

4Cabinet（单数）在这种情况下是一个一致性吸引子。

Model	Number of attractors						
	0	1	2	3	4	5	Total
Previous best results (Yogatama et al., 2018):							
Best Stack-RNN	*0.994*	0.979	0.965	0.935	0.916	0.880	0.992
Best LSTM	0.993	0.972	0.950	0.922	0.900	0.842	0.991
Best Attention	**0.994**	**0.977**	0.959	0.929	0.907	0.842	**0.992**
Our results:							
Transformer	0.973	0.941	0.932	0.917	0.901	0.883	0.962
Universal Transformer	0.993	0.971	**0.969**	0.940	0.921	0.892	**0.992**
UT w/ ACT	**0.994**	0.969	0.967	**0.944**	**0.932**	**0.907**	**0.992**
Δ (UT w/ ACT - Best)	0	-0.008	0.002	0.009	0.016	0.027	-

表 2：主谓数一致性预测任务的准确率（越高越好）。

Model	LM Perplexity & (Accuracy)			RC Accuracy		
	control	dev	test	control	dev	test
Neural Cache (Grave et al., 2016)	**129**	139	-	-	-	-
Dhingra et al. Dhingra et al. (2018)	-	-	-	-	-	0.5569
Transformer	142 (0.19)	5122 (0.0)	7321 (0.0)	0.4102	0.4401	0.3988
LSTM	138 (0.23)	4966 (0.0)	5174 (0.0)	0.1103	0.2316	0.2007
UT *base*, 6 steps (fixed)	131 (0.32)	279 (0.18)	319 (0.17)	**0.4801**	0.5422	0.5216
UT w/ dynamic halting	130 (0.32)	**134** (0.22)	**142** (0.19)	0.4603	**0.5831**	**0.5625**
UT *base*, 8 steps (fixed)	129(0.32)	192 (0.21)	202 (0.18)	-	-	-
UT *base*, 9 steps (fixed)	**129(0.33)**	214 (0.21)	239 (0.17)	-	-	-

表 3：LAMBADA 语言建模（LM）困惑度（越低越好），括号内为准确率（越高越好），以及阅读理解（RC）准确率结果（越高越好）。“-”表示该设置下没有报告结果。

建模，并测试模型在预测目标词时融入更广泛话语和长期上下文的能力。

该任务在两种设置下进行评估：作为语言建模（标准设置）和作为阅读理解。在前一种（更具挑战性的）情况下，模型仅在训练数据上进行下一词预测训练，并在测试时评估目标词（即模型被训练来预测所有词，而非专门针对具有挑战性的目标词）。在后一种设置中，由 Chu 等人（2017）引入，目标句子（去掉最后一个词）被用作查询，从上下文句子中选择目标词。注意，目标词在 81% 的情况下出现在上下文中，这使得该设置简单得多。然而，在剩余的 19% 的情况下，该任务是不可能完成的。

结果如表 3 所示。通用变换器在语言建模和阅读理解设置中均取得了最先进的结果，优于长短期记忆网络和普通变换器。注意，控制集的构建方式与 LAMBADA 开发集和测试集类似，但未进行任何过滤，因此在该集上取得良好结果显示了模型在标准语言建模中的优势。

我们最佳的固定步数通用变换器结果使用了 6 步。然而，在测试数据上，具有动态停止的最佳通用变换器在所有位置和示例上的平均步数为 8.2±2.1 。为了查看动态模型是否仅仅因为采取了更多步数而表现更好，我们分别训练了两个固定步数为 8 步和 9 步的通用变换器模型（见最后两行）。有趣的是，这两个模型与 6 步模型相比取得了更好的结果，但并未优于具有动态停止的通用变换器。这使我们相信，动态停止可能通过激励某些输入符号使用更少的步数，同时允许其他符号进行更多计算，从而作为模型的一种有用的正则化器。

3.4 算法任务

我们在三个算法任务上训练了通用变换器，即复制、反转和（整数）加法，所有任务均基于由十进制符号（‘0’-‘9’）组成的字符串。在所有实验中，我们在长度为 40 的序列上训练模型，并在长度为 400 的序列上进行评估（Kaiser & Sutskever, 2016）。我们

Model	Copy		Reverse		Addition	
	char-acc	seq-acc	char-acc	seq-acc	char-acc	seq-acc
LSTM	0.45	0.09	0.66	0.11	0.08	0.0
Transformer	0.53	0.03	0.13	0.06	0.07	0.0
Universal Transformer	0.91	0.35	0.96	0.46	0.34	0.02
Neural GPU*	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**

表 4：算法任务上的准确率（越高越好）。 $*$ 注意，神经 GPU 通过特殊课程训练以获得完美结果，而其他模型则未使用任何课程进行训练。

Model	Copy		Double		Reverse	
	char-acc	seq-acc	char-acc	seq-acc	char-acc	seq-acc
LSTM	0.78	0.11	0.51	0.047	0.91	0.32
Transformer	0.98	0.63	0.94	0.55	0.81	0.26
Universal Transformer	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**

表 5：记忆长短期记忆任务上的字符级（字符准确率）和序列级准确率（序列准确率）结果，最大长度为 55。

Model	Program		Control		Addition	
	char-acc	seq-acc	char-acc	seq-acc	char-acc	seq-acc
LSTM	0.53	0.12	0.68	0.21	0.83	0.11
Transformer	0.71	0.29	0.93	0.66	**1.0**	**1.0**
Universal Transformer	**0.89**	**0.63**	**1.0**	**1.0**	**1.0**	**1.0**

表 6：在最大嵌套深度为 2、长度为 5 的程序评估 LTE 任务上的字符级（字符准确率）和序列级准确率（序列准确率）结果。

使用从随机偏移位置开始的训练通用变换器，以进一步鼓励模型学习位置相对变换。结果如表 4 所示。通用变换器在所有三个任务上均大幅超越长短期记忆网络和标准变换器。神经 GPU 在该任务上报告了完美结果（Kaiser & Sutskever, 2016），然而我们注意到该结果需要特殊的基于课程学习的训练协议，而其他模型未使用该协议。

3.5 学习执行（LTE）

作为另一类序列到序列学习问题，我们还在任务上评估了通用变换器，这些任务表明模型学习执行计算机程序的能力，如（Zaremba & Sutskever, 2015）所提出。这些任务包括程序评估任务（程序、控制、加法）和记忆任务（复制、加倍、反转）。

我们使用（Zaremba & Sutskever, 2015）中讨论的混合策略生成数据集。与（Zaremba & Sutskever, 2015）不同，我们在训练期间不使用任何课程学习策略，并且在测试时不使用目标序列。表 5 和表 6 分别展示了长短期记忆网络模型、变换器和通用变换器在程序评估和记忆任务上的性能。通用变换器在所有记忆任务中均取得满分，并且在所有程序评估任务中也大幅超越长短期记忆网络和变换器。

3.6 机器翻译

我们在 WMT 2014 英语-德语翻译任务上训练了通用变换器，使用与（Vaswani et al., 2017）相同的设置，以评估其在大规模序列到序列任务上的性能。结果总结在表 7 中。具有全连接循环转移函数（而非可分离卷积）且不使用自适应计算时间的通用变换器，在参数数量大致相同的情况下，比变换器提高了 0.9 BLEU，比加权变换器提高了 0.5 BLEU（Ahmed et al., 2017）。

Model	BLEU
Universal Transformer *small*	26.8
Transformer *base* (Vaswani et al., 2017)	28.0
Weighted Transformer *base* (Ahmed et al., 2017)	28.4
Universal Transformer *base*	**28.9**

表 7：在 WMT14 英语-德语翻译任务上的机器翻译结果，在 8 块 P100 GPU 上以可比的训练设置进行训练。所有基础结果具有相同的参数数量。

4 讨论

当以固定步数运行时，通用变换器（Universal Transformer）等价于一个多层变换器，其所有层之间共享参数。这在部分程度上类似于递归变换器（Recursive Transformer），后者在深度方向上共享自注意力层的权重（Gulcehre et al., 2018）[5]. 。然而，由于每个符号的循环转移函数可以被应用任意次数，另一种可能更具信息量的描述方式是将通用变换器视为一组并行的循环神经网络（RNN）块（每个符号对应一个，参数共享），这些块同时演化每个符号的隐藏状态，每一步通过关注前一步的隐藏状态序列来生成当前状态。通过这种方式，它与神经图形处理器（Neural GPU）（Kaiser & Sutskever, 2016）和神经图灵机（Neural Turing Machine）（Graves et al., 2014）等架构相关联。因此，通用变换器保留了原始前馈变换器模型的高计算效率，同时增加了循环神经网络特有的循环归纳偏置。此外，通过使用动态停止机制，通用变换器可以根据输入数据选择处理步数。

通用变换器与其他序列模型之间的联系从架构上显而易见：如果我们将循环步数限制为一步，它就是一个变换器。但更有趣的是考虑通用变换器与循环神经网络以及其他在时间维度上进行循环的网络之间的关系。表面上看，这些模型可能密切相关，因为它们同样具有循环特性。但存在一个关键区别：像循环神经网络这样的时间循环模型无法在循环步骤中访问记忆。这使得它们在计算上更类似于自动机，因为循环部分唯一可用的记忆是一个固定大小的状态向量。相比之下，通用变换器可以关注整个前一层，从而能够在循环步骤中访问记忆。

在内存充足的条件下，通用变换器具备计算通用性——即它属于能够模拟任意图灵机的模型类别，从而弥补了标准变换器模型 6 的不足。除了理论上的吸引力之外，我们的结果表明，这种增强的表达能力还在多个具有挑战性的序列建模任务上带来了准确率的提升。这缩小了在大规模任务（如机器翻译）上具有竞争力的实用序列模型与可通过梯度下降训练以执行算法任务的计算通用模型（如神经图灵机或神经图形处理器，Graves 等人，2014；Kaiser 与 Sutskever，2016）之间的差距。

为证明这一点，我们可以将神经图形处理器归约为通用变换器。忽略解码器并将自注意力模块（即带残差连接的自注意力）参数化为恒等函数，我们假设转移函数为卷积。若将总循环步数 T 设为与输入长度相等，则恰好得到神经图形处理器。注意，最后一步正是通用变换器与普通变换器的关键区别所在，后者的深度无法随输入规模动态扩展。通用变换器与神经图灵机之间也存在类似关系，后者每步的单次读写操作可由通用变换器的全局并行表示修正来表达。然而，与这些仅在算法任务上表现良好的模型相比，通用变换器在现实自然语言任务（如 LAMBADA 和机器翻译）上也取得了具有竞争力的结果。

另一种相关的模型架构是端到端记忆网络（Sukhbaatar 等人，2015）。然而，与端到端记忆网络不同，通用变换器使用的记忆对应于其输入或输出中各个位置对齐的状态。此外，通用变换器遵循编码器-解码器配置，并在大规模序列到序列任务中取得了具有竞争力的性能。

5 注意，在通用变换器中，自注意力和转移权重在各层之间是共享的。6 附录 B 说明了通用变换器在计算能力上如何强于标准变换器。

5 结论

本文介绍了通用变换器（Universal Transformer），它是变换器（Transformer）模型的一种推广，扩展了其理论能力，并在广泛的具有挑战性的序列建模任务上取得了最先进的结果，例如语言理解以及各种算法任务，从而解决了标准变换器的一个关键不足。通用变换器将以下关键特性结合到一个模型中：

权重共享：遵循卷积神经网络（CNN）和循环神经网络（RNN）中权重共享背后的直觉，我们用一种简单的权重共享形式扩展了变换器，在归纳偏置和模型表达能力之间取得了有效的平衡，我们在小规模和大规模实验中都广泛展示了这一点。

条件计算：在构建计算通用机器的目标中，我们通过最近引入的机制为通用变换器配备了停止或继续计算的能力，与固定深度的通用变换器相比，该机制显示出更强的结果。

我们对时间并行序列模型的最新发展感到兴奋。通过在处理深度上增加计算能力和循环，我们希望在此处介绍的基本通用变换器之外取得进一步改进，将帮助我们构建更强大、数据效率更高且能超越当前最先进水平泛化的学习算法。

用于训练和评估通用变换器的代码可在 https://github.com/tensorflow/tensor2tensor 获取（Vaswani 等人，2018）。

致谢 我们感谢 Ashish Vaswani、Douglas Eck 和 David Dohan 提出的富有成效的意见和启发。

参考文献

Karim Ahmed, Nitish Shirish Keskar, and Richard Socher. Weighted transformer network for machine translation. *arXiv preprint arXiv:1711.02132*, 2017.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL http://arxiv.org/abs/1607.06450.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014. URL http://arxiv.org/abs/1409.0473.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014. URL http://arxiv.org/abs/1406.1078.

Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

Zewei Chu, Hai Wang, Kevin Gimpel, and David McAllester. Broad context language modeling as reading comprehension. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, volume 2, pp. 52–57, 2017.

Bhuwan Dhingra, Zhilin Yang, William W Cohen, and Ruslan Salakhutdinov. Linguistic knowledge as memory for recurrent neural networks. *arXiv preprint arXiv:1703.02620*, 2017.

Bhuwan Dhingra, Qiao Jin, Zhilin Yang, William W Cohen, and Ruslan Salakhutdinov. Neural models for reasoning over multiple mentions using coreference. *arXiv preprint arXiv:1804.05922*, 2018.

Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *CoRR*, abs/1705.03122, 2017. URL http://arxiv.org/abs/1705.03122.

Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. *arXiv preprint arXiv:1612.04426*, 2016.

Alex Graves. Generating sequences with recurrent neural networks. *CoRR*, abs/1308.0850, 2013. URL http://arxiv.org/abs/1308.0850.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *CoRR*, abs/1410.5401, 2014. URL http://arxiv.org/abs/1410.5401.

Caglar Gulcehre, Misha Denil, Mateusz Malinowski, Ali Razavi, Razvan Pascanu, Karl Moritz Hermann, Peter Battaglia, Victor Bapst, David Raposo, Adam Santoro, et al. Hyperbolic attention networks. *arXiv preprint arXiv:1805.09786*, 2018.

Mikael Henaff, Jason Weston, Arthur Szlam, Antoine Bordes, and Yann LeCun. Tracking the world state with recurrent entity networks. *arXiv preprint arXiv:1612.03969*, 2016.

Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. *A Field Guide to Dynamical Recurrent Neural Networks*, 2003.

A. Joulin and T. Mikolov. Inferring algorithmic patterns with stack-augmented recurrent nets. In *Advances in Neural Information Processing Systems, (NIPS)*, 2015.

Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1511.08228.

Łukasz Kaiser, Aidan N. Gomez, and Francois Chollet. Depthwise separable convolutions for neural machine translation. *CoRR*, abs/1706.03059, 2017. URL http://arxiv.org/abs/1706.03059.

Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International Conference on Machine Learning*, pp. 1378–1387, 2016.

Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

Tal Linzen, Emmanuel Dupoux, and Yoav Goldberg. Assessing the ability of lstms to learn syntax-sensitive dependencies. *Transactions of the Association of Computational Linguistics*, 4(1):521–535, 2016.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, volume 1, pp. 1525–1534, 2016.

Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016. URL https://arxiv.org/pdf/1606.01933.pdf.

Jack Rae, Jonathan J Hunt, Ivo Danihelka, Timothy Harley, Andrew W Senior, Gregory Wayne, Alex Graves, and Tim Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes. In *Advances in Neural Information Processing Systems*, pp. 3621–3629, 2016.

Minjoon Seo, Sewon Min, Ali Farhadi, and Hannaneh Hajishirzi. Query-reduction networks for question answering. *arXiv preprint arXiv:1606.04582*, 2016.

Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1): 1929–1958, 2014.

Sainbayar Sukhbaatar, arthur szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 28*, pp. 2440–2448. Curran Associates, Inc., 2015. URL http://papers.nips.cc/paper/5846-end-to-end-memory-networks.pdf.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pp. 3104–3112, 2014. URL http://arxiv.org/abs/1409.3215.

Ke Tran, Arianna Bisazza, and Christof Monz. The importance of being recurrent for modeling hierarchical structure. In *Proceedings of NAACL'18*, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, 2017. URL http://arxiv.org/abs/1706.03762.

Ashish Vaswani, Samy Bengio, Eugene Brevdo, Francois Chollet, Aidan N. Gomez, Stephan Gouws, Llion Jones, Łukasz Kaiser, Nal Kalchbrenner, Niki Parmar, Ryan Sepassi, Noam Shazeer, and Jakob Uszkoreit. Tensor2tensor for neural machine translation. *CoRR*, abs/1803.07416, 2018.

Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015.

Dani Yogatama, Yishu Miao, Gabor Melis, Wang Ling, Adhiguna Kuncoro, Chris Dyer, and Phil Blunsom. Memory architectures in recurrent neural network language models. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=SkFqf0lAZ.

Wojciech Zaremba and Ilya Sutskever. Learning to execute. *CoRR*, abs/1410.4615, 2015. URL http://arxiv.org/abs/1410.4615.

附录 A 通用变换器的详细模式

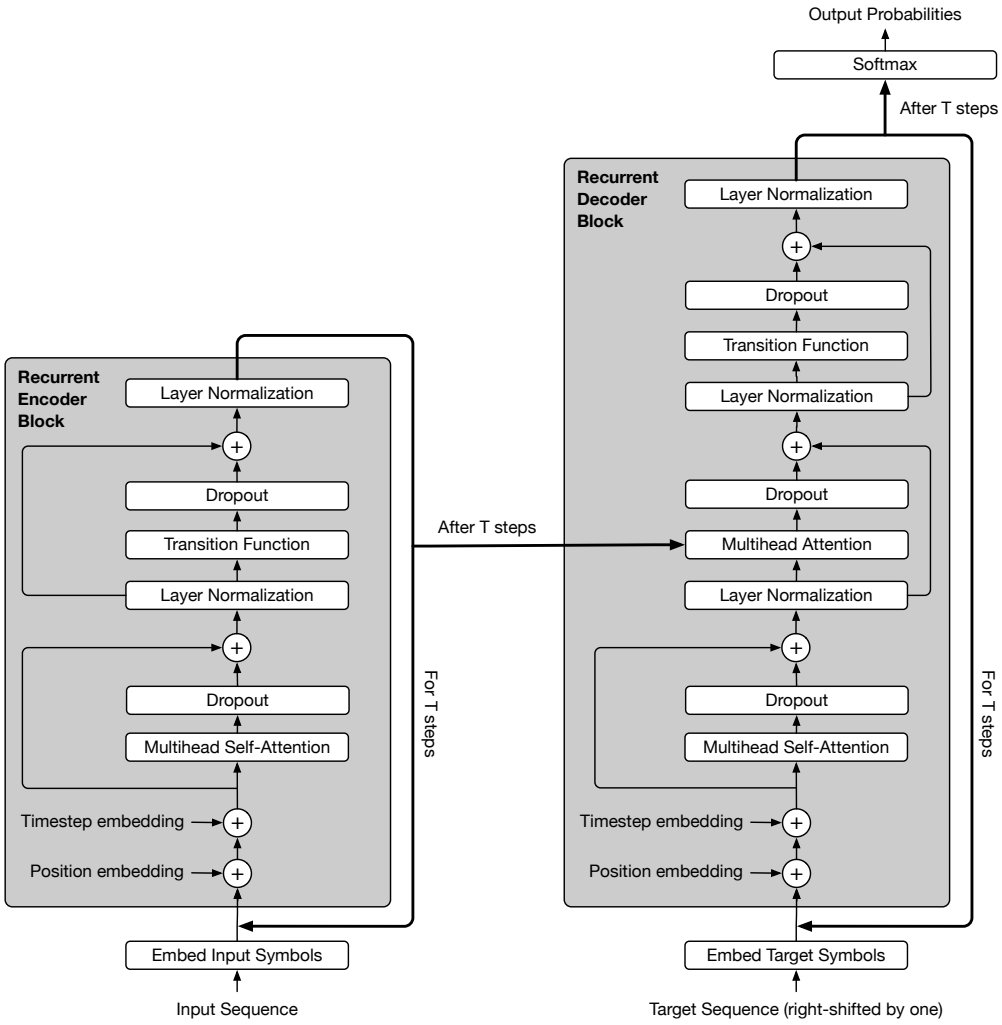
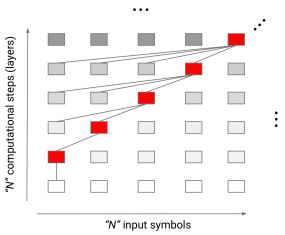


图 4：带有位置和步长嵌入以及随机失活和层归一化的通用变换器。

附录 B 关于通用变换器与变换器计算能力的讨论

就计算能力而言，变换器与通用变换器之间的关键区别在于顺序计算步骤的数量（即深度）。标准变换器执行的总操作数随输入规模而变化，但顺序操作的数量是恒定的，与输入规模无关，仅由层数决定。假设精度有限，这一特性意味着标准变换器在计算上不可能是通用的。然而，当选择步数作为输入长度的函数时，通用变换器不受此限制。注意，无论是否采用自适应计算时间，这一点都成立，但确实假设步数非常数，即使可能是确定性的。通用变换器通过在顺序计算步骤之间共享权重，使得训练后可以动态改变步数。

一个直观的例子是那些执行时需要顺序处理每个输入元素的函数。在这种情况下，对于任意给定的深度 T，都可以构造一个长度为 $N > T$ 的输入序列，使得标准变换器无法正确处理。然而，通过选择与输入长度相关的适当顺序步骤，通用变换器、循环神经网络或神经 GPU 可以执行这样的函数。



附录 C 具有动态停止机制的通用变换器

我们在 TensorFlow 中基于 ACT（Graves, 2016）实现动态停止，具体如下。在具有动态停止的 UT 的每一步中，我们给定停止概率、余数、到该点为止的更新次数以及前一个状态（均初始化为零），以及一个介于 0 和 1 之间的标量阈值（超参数）。然后，我们计算每个位置的新状态，并根据每个位置的状态计算新的逐位置停止概率。UT 随后决定对某些超过阈值的位置停止，并更新其他位置的状态，直到模型对所有位置停止或达到预定义的最大步数：

```
# 当 - 循环在此谓词为 FALSE 时停止
# i.e. all ((probability < threshold) & (counter < max_steps)) are false
def should_continue(u0, u1, halting_probability, u2, n_updates, u3):
return tf.reduce_any(
            tf.logical_and(
                tf.less(halting_probability, threshold),
                tf.less(n_updates, max_steps)))
# Do while loop iterations until predicate above is false
(_, _, _, remainder, n_updates, new_state) = tf.while_loop(
    should_continue, ut_with_dynamic_halting, (state,
    step, halting_probability, remainders, n_updates, previous_state))
```

Listing 1: UT with dynamic halting.

以下展示了每一步中的计算：

```
def ut_with_dynamic_halting(state, step, halting_probability,
                            remainders, n_updates, previous_state):
    # Calculate the probabilities based on the state
    p = common_layers.dense(state, 1, activation=tf.nn.sigmoid,
        use_bias=True)
    # Mask for inputs which have not halted yet
    still_running = tf.cast(
        tf.less(halting_probability,1.0), tf.float32)
    # Mask of inputs which halted at this step
    new_halted = tf.cast(
        tf.greater(halting_probability + p * still_running, threshold),
            tf.float32) * still_running
    # Mask of inputs which haven't halted, and didn't halt this step
    still_running = tf.cast(
        tf.less_equal(halting_probability + p * still_running,
            threshold), tf.float32) * still_running
    # Add the halting probability for this step to the halting
    # probabilities for those inputs which haven't halted yet
    halting_probability += p * still_running
    # Compute remainders for the inputs which halted at this step
    remainders += new_halted * (1 - halting_probability)
    # Add the remainders to those inputs which halted at this step
    halting_probability += new_halted * remainders
    # Increment n_updates for all inputs which are still running
    n_updates += still_running + new_halted
    # Compute the weight to be applied to the new state and output:
    #   0 when the input has already halted,
    #   p when the input hasn't halted yet,
    #   the remainders when it halted this step.
    update_weights = tf.expand_dims(p * still_running +
                                    new_halted * remainders, -1)
    # Apply transformation to the state
    transformed_state = transition_function(self_attention(state))
    # Interpolate transformed and previous states for non-halted inputs
    new_state = ((transformed_state * update_weights) +
                 (previous_state * (1 - update_weights)))
    step += 1
    return (transformed_state, step, halting_probability,
            remainders, n_updates, new_state)
```

Listing 2: Computations in each step of the UT with dynamic halting.

附录 D 部分 TASKS/DATASETS 的描述

在此，我们提供关于 bAbI、主谓一致、LAMBADA 语言建模以及学习执行（LTE）任务的一些额外细节。

D.1 BABI 问答

bAbI 问答数据集（Weston et al., 2015）包含 20 个不同的合成任务 [7] 。其目标是每个任务测试语言理解和推理的一个独特方面，包括：从故事中的支持事实进行推理、回答真/假类型问题、计数、理解否定和不确定知识、理解共指、时间推理、位置和大小推理、路径查找以及理解动机（每个任务的示例请参见（Weston et al., 2015）中的表 1）。

该数据集有两个版本，一个包含 1k 训练示例，另一个包含 10k 示例。模型必须具有数据效率，才能仅使用 1k 训练示例取得良好结果。此外，最初的想法是应该在所有任务上评估单一模型（而不是针对每个任务进行调整），即表 1 和附录 E 中表格所展示的联合训练设置。

D.2 主谓一致

主谓一致任务旨在预测英语句子中主语与动词之间的数的一致性。成功完成该任务是一个强有力的指标，表明模型能够学习近似句法结构，因此林岑等人（Linzen et al., 2016）提出将其作为评估不同模型捕捉自然语言中层次结构能力的代理任务。

林岑等人（Linzen et al., 2016）为训练该任务提出了两种实验设置：1）以语言建模目标进行训练，即预测下一个词；2）作为二分类任务，即给定句子预测动词的数。本文采用语言建模目标，这意味着我们为模型提供隐式监督，并根据正确动词形式相对于错误动词形式的排序准确率进行评估。

在该任务中，为了设置不同难度级别，使用了“一致性吸引词”，即一个或多个与主语数相反的中间名词，目的是迷惑模型。在这种情况下，模型需要正确识别与给定动词对应的句法主语的中心词，并忽略中间的吸引词，以预测该动词的正确形式。以下是该任务的一些示例，其中主语和对应动词以粗体显示，一致性吸引词以下划线标出：

```
No attractor:       The boy smiles.
One attractor:      The number of men is not clear.
Two attractors:     The ratio of men to women is not clear.
Three attractors:   The ratio of men to women and children is not clear.
```

D.3 LAMBADA 语言建模

LAMBADA 任务（Paperno et al., 2016）是一项宽泛上下文语言建模任务。在该任务中，给定一段叙述性文本，目标是预测该段落最后一句（目标句）的最后一个词（目标词）。这些段落经过专门挑选，使得人类受试者在看到长段落时能够轻松猜出最后一个词，但如果只看到目标词之前的目标句则无法猜出 [8]. 。以下是数据集中的一个示例：

上下文: “是的，我以为我会失去这个孩子。”“我也很害怕，”他说道，眼中满是真诚。“你害怕？”“是的，当然。你为什么这么问？”“这个孩子并不是计划中的。”目标句：“你真的认为我会想让你有一个_____？”目标词：流产

LAMBADA 任务的目标是在给定整个段落（即上下文加上目标句）的情况下预测目标词。还提供了一个“对照集”，该对照集是通过随机采样与构建 LAMBADA 所用段落形状和大小相同的段落而构建的，但未以任何方式对其进行过滤。对照

[7] https://research.fb.com/downloads/babi

[8] http://clic.cimec.unitn.it/lambada/appendix_onefile.pdf

集用于在 LAMBADA 任务测试之前评估模型在标准语言建模上的表现，从而确保在 LAMBADA 任务上的低性能不能简单地归因于语言建模能力差。

该任务在两种设置下进行评估：作为语言建模（标准设置）和作为阅读理解。在前一种（更具挑战性的）情况下，模型仅在训练数据上进行下一个词预测的训练，并在测试时对目标词进行评估（即模型被训练来预测所有词，而不是专门针对具有挑战性的目标词）。在本文中，我们报告了通用变换器在两种设置下的结果。

D.4 学习执行（LTE）

LTE 是一组任务，用于指示模型学习执行计算机程序的能力，由 Zaremba & Sutskever（2015）提出。这些任务包括两个子集：1）程序评估任务（程序、控制和加法），旨在评估模型理解数值运算、if 语句、变量赋值、运算组合性等能力，以及 2）记忆任务（复制、加倍和反转）。

程序评估任务的难度由其长度和嵌套参数化。长度参数是程序中出现的整数的位数（因此整数从 [1，长度] 中均匀选择），嵌套参数是我们允许将操作相互组合的次数。嵌套值越高，生成的程序解析树越深。例如，这里有一个程序，其生成参数为长度 = 4 和嵌套 = 3。

Input:

```
j=8584
for x in range(8):
  j+=920
b=(1500+j)
print((b+7567))
```

Target:

```
25011
```

附录 E BABI 详细结果

Best seed run for each task (out of 10 runs)				
Task id	10K		1K	
	train single	train joint	train single	train joint
1	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.5
3	0.4	1.2	3.7	5.4
4	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.5
6	0.0	0.0	0.0	0.5
7	0.0	0.0	0.0	3.2
8	0.0	0.0	0.0	1.6
9	0.0	0.0	0.0	0.2
10	0.0	0.0	0.0	0.4
11	0.0	0.0	0.0	0.1
12	0.0	0.0	0.0	0.0
13	0.0	0.0	0.0	0.6
14	0.0	0.0	0.0	3.8
15	0.0	0.0	0.0	5.9
16	0.4	1.2	5.8	15.4
17	0.6	0.2	32.0	42.9
18	0.0	0.0	0.0	4.1
19	2.8	3.1	47.1	68.2
20	0.0	0.0	2.4	2.4
avg err	0.21	0.29	4.55	7.78
failed	0	0	3	5

所有种子（10 次运行）的平均值（ ± var）任务 ID 10K 1K 训练 单任务 训练 联合 训练 单任务 训练 联合				
1	0.0 ±0.0	0.0 ±0.0	0.2 ±0.3	0.1 ±0.2
2	0.2 ±0.4	1.7 ±2.6	3.2 ±4.1	4.3 ±11.6
3	1.8 ±1.8	4.6 ±7.3	9.1 ±12.7	14.3 ±18.1
4	0.1 ±0.1	0.2 ±0.1	0.3 ±0.3	0.4 ±0.6
5	0.2 ±0.3	0.8 ±0.5	1.1 ±1.3	4.3 ±5.6
6	0.1 ±0.2	0.1 ±0.2	1.2 ±2.1	0.8 ±0.4
7	0.3 ±0.5	1.1 ±1.5	0.0 ±0.0	4.1 ±2.9
8	0.3 ±0.2	0.5 ±1.1	0.1 ±0.2	3.9 ±4.2
9	0.0 ±0.0	0.0 ±0.0	0.1 ±0.1	0.3 ±0.3
10	0.1 ±0.2	0.5 ±0.4	0.7 ±0.8	1.3 ±1.6
11	0.0 ±0.0	0.1 ±0.1	0.4 ±0.8	0.3 ±0.9
12	0.2 ±0.1	0.4 ±0.4	0.6 ±0.9	0.3 ±0.4
13	0.2 ±0.5	0.3 ±0.4	0.8 ±0.9	1.1 ±0.9
14	1.8 ±2.6	1.3 ±1.6	0.1 ±0.2	4.7 ±5.2
15	2.1 ±3.4	1.6 ±2.8	0.3 ±0.5	10.3 ±8.6
16	1.9 ±2.2	0.9 ±1.3	9.1 ±8.1	34.1 ±22.8
17	1.6 ±0.8	1.4 ±3.4	43.7 ±18.6	51.1 ±12.9
18	0.3 ±0.4	0.7 ±1.4	2.3 ±3.6	12.8 ±9.0
19	3.4 ±4.0	6.1 ±7.3	50.2 ±8.4	73.1 ±23.9
20	0.0 ±0.0	0.0 ±0.0	3.2 ±2.5	2.6 ±2.8
avg	0.73 ±0.89	1.12 ±1.62	6.34 ±3.32	11.21 ±6.65

附录 F BABI 注意力可视化

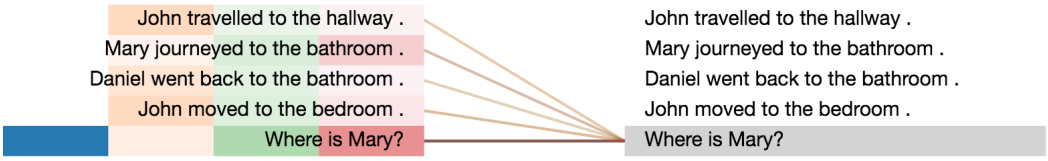
本文展示了在 bAbI 任务上几个示例的注意力分布可视化。注意力权重的可视化基于不同时间步，涉及故事中所有事实和问题上的不同头。左侧不同颜色的条形表示基于不同头（共 4 个头）的注意力权重。

任务 1 的一个示例：（需要一个支持性事实来解决）

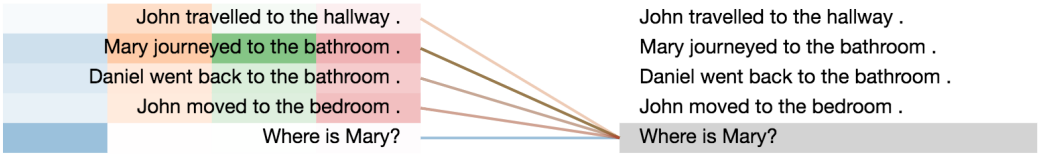
故事：

约翰走到走廊。玛丽走到浴室。丹尼尔回到浴室。约翰移动到卧室。

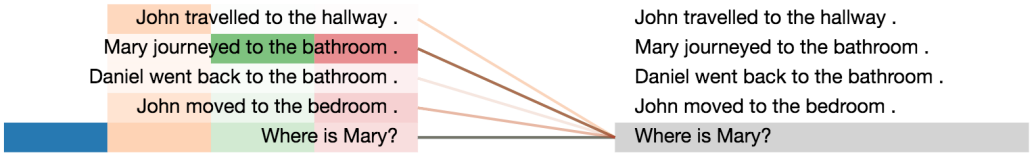
问题：玛丽在哪里？模型输出：浴室



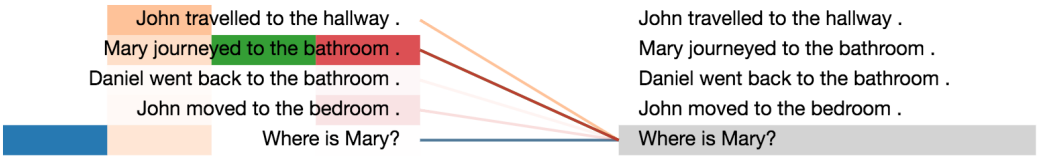
(a) Step 1



(b) Step 2



(c) Step 3



(d) Step 4

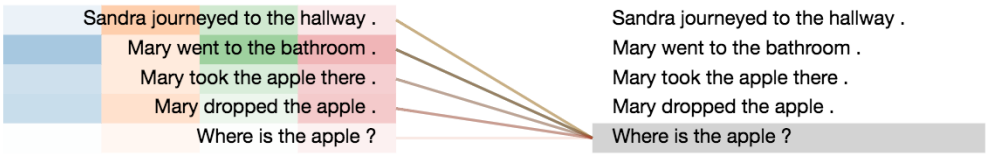
图 5：编码问题“玛丽在哪里？”时注意力分布的可视化。

任务 2 中的一个示例：（需要两个支持性事实才能解决）

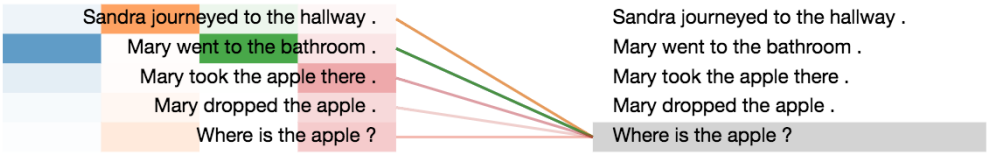
故事：

桑德拉走到走廊。玛丽去了浴室。玛丽在那里
拿了苹果。玛丽把苹果掉了。

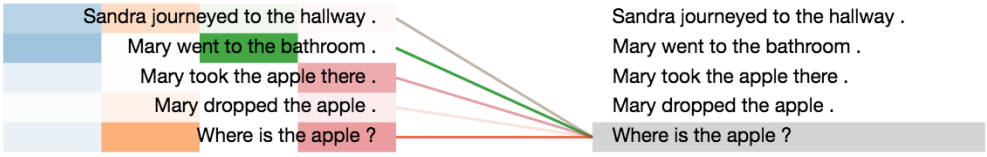
问题：苹果在哪里？模型输出：浴室



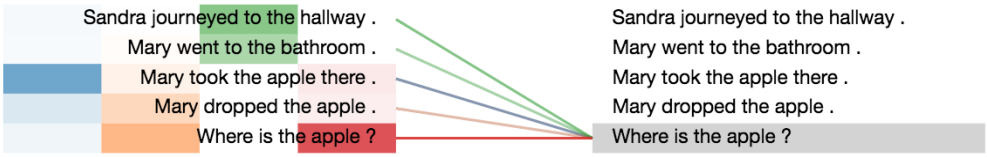
(a) Step 1



(b) Step 2



(c) Step 3



(d) Step 4

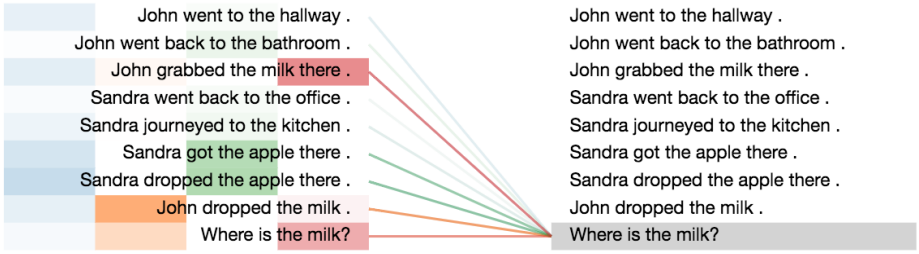
图 6：编码问题“苹果在哪里？”时的注意力分布可视化。

任务 2 中的一个示例：（需要两个支持性事实才能解决）

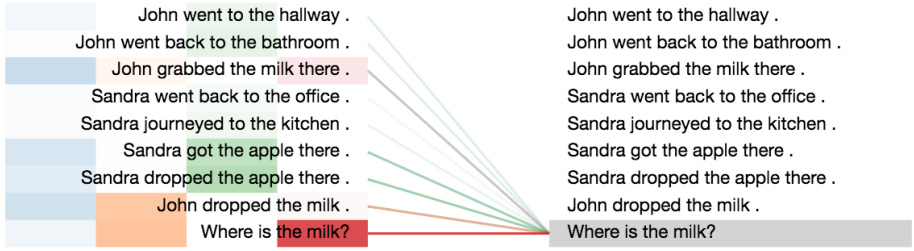
故事：

约翰走到走廊。约翰回到浴室。约翰在那里拿了牛奶。桑德拉回到办公室。桑德拉走到厨房。桑德拉在那里拿了苹果。桑德拉把苹果掉在那里。约翰把牛奶掉了。

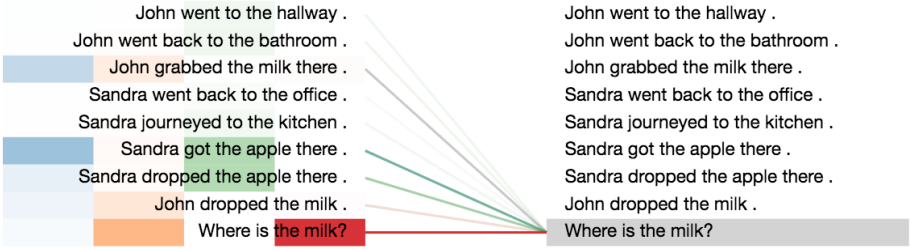
问题：牛奶在哪里？模型输出：浴室



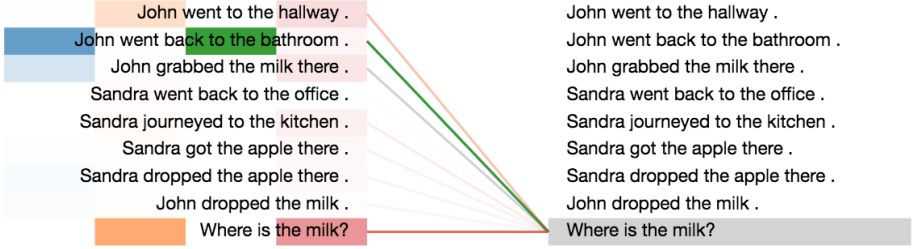
(a) Step 1



(b) Step 2



(c) Step 3



(d) Step 4

图 7：编码问题“牛奶在哪里？”时的注意力分布可视化。

任务 3 中的一个示例：（需要三个支持性事实才能解决）

故事：玛丽拿到了牛奶。

约翰搬到了卧室。丹尼尔去了办公室。约翰在那里拿到了苹果。约翰拿到了足球。约翰去了花园。玛丽留下了牛奶。约翰留下了足球。丹尼尔搬到了花园。丹尼尔拿到了足球。玛丽搬到了走廊。玛丽去了厨房。约翰把苹果放在那里。约翰捡起了苹果。桑德拉搬到了走廊。丹尼尔把足球留在那里。丹尼尔拿走了足球。约翰去了厨房。丹尼尔扔下了足球。约翰扔下了苹果。约翰拿到了苹果。约翰去了办公室。桑德拉回到了卧室。桑德拉拿走了牛奶。约翰去了浴室。约翰去了办公室。桑德拉留下了牛奶。玛丽去了卧室。玛丽搬到了办公室。约翰去了走廊。桑德拉搬到了花园。玛丽搬到了厨房。丹尼尔拿走了足球。玛丽去了卧室。玛丽在那里拿到了牛奶。玛丽扔掉了牛奶。约翰去了花园。约翰把苹果扔在那里。

问题：苹果在浴室之前在哪里？模型输出：办公室



(e) Step 1



(f) Step 2



(g) Step 3



(h) Step 4

图 7：编码问题“苹果在浴室之前在哪里？”时的注意力分布可视化。